# RedHat OpenShift - Développement niveau 1 - Applications de conteneurisation

Utopios® Tous droits réservés - lot 4-002 20/03/2026

# Rappel Jour 1 et Programme Jour 2

## Jour 1 — Acquis :

- Architecture OpenShift, CLI, Console Web
- Projets, RBAC, SCC, Quotas

## Jour 2 — Programme :

| Module | Thème | Durée |
|---|---|---|
| 4 | Création et déploiement d'applications | 2h |
| 5 | Gestion des conteneurs et des images | 2h |
| 6 | Optimisation des services et microservices | 2h |

Utopios® Tous droits réservés - lot 4-002 20/03/2026

# Module 4

## Création et Déploiement d'Applications avec OpenShift

*Durée : 2 heures*

# Méthodes de déploiement sur OpenShift

OpenShift offre plusieurs stratégies pour déployer des applications :

| Méthode | Description | Cas d'usage |
|---|---|---|
| **oc new-app** | Déploiement rapide depuis source/image | Prototypage, dev |
| **Templates** | Modèles paramétrables | Standardisation |
| **S2I** | Build automatique depuis le code source | CI/CD intégré |
| **Helm Charts** | Gestionnaire de packages K8s | Apps complexes |
| **Operators** | Gestion du cycle de vie complet | Apps stateful |
| **YAML/JSON** | Manifestes déclaratifs | Contrôle total |

Utopios® Tous droits réservés - lot 4-002 20/03/2026

# Déploiement avec `oc new-app`

```
# Depuis une image Docker Hub
oc new-app nginx:latest --name=mon-nginx

# Depuis un dépôt Git (détection automatique du langage)
oc new-app https://github.com/sclorg/nodejs-ex.git \
  --name=mon-app-node

# Depuis une image avec variables d'environnement
oc new-app mysql:8.0 \
  --name=ma-db \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=formation \
  -e MYSQL_USER=user \
  -e MYSQL_PASSWORD=password

# Vérifier le déploiement
oc get pods -w
```

# Exposition des applications — Routes

```
# Créer une route HTTP
oc expose svc/mon-nginx
oc get route

# Créer une route HTTPS (edge termination)
oc create route edge mon-nginx-secure \
  --service=mon-nginx \
  --hostname=mon-app.apps-crc.testing

# Route avec certificat personnalisé
oc create route edge mon-nginx-tls \
  --service=mon-nginx \
  --cert=tls.crt \
  --key=tls.key \
```

# Les Templates OpenShift

Un Template est un modèle paramétrable pour créer des ressources :

```yaml
apiVersion: template.openshift.io/v1
kind: Template
metadata:
  name: webapp-template
  annotations:
    description: "Template pour application web"
parameters:
  - name: APP_NAME
    description: "Nom de l'application"
    required: true
  - name: IMAGE_TAG
    description: "Tag de l'image"
    value: "latest"
  - name: REPLICAS
    description: "Nombre de réplicas"
```

# Template — Objets (suite)

```yaml
objects:
  - apiVersion: apps/v1
    kind: Deployment
    metadata:
      name: ${APP_NAME}
    spec:
      replicas: ${{REPLICAS}}
      selector:
        matchLabels:
          app: ${APP_NAME}
      template:
        metadata:
          labels:
            app: ${APP_NAME}
        spec:
          containers:
            - name: ${APP_NAME}
              image: "nginx:${IMAGE_TAG}"
              ports:
                - containerPort: 8080
              resources:
                requests:
                  memory: "128Mi"
                  cpu: "100m"
```

# Utilisation des Templates

```
# Charger un template dans le cluster
oc create -f webapp-template.yaml

# Lister les templates disponibles
oc get templates
oc get templates -n openshift  # Templates globaux

# Voir les paramètres d'un template
oc process webapp-template --parameters

# Instancier un template
oc process webapp-template \
  -p APP_NAME=mon-webapp \
  -p IMAGE_TAG=1.21 \
  -p REPLICAS=3 | oc apply -f -

# Ou directement avec new-app
```

# Source-to-Image (S2I)

**S2I** construit automatiquement des images depuis le code source :

```
┌──────────┐   ┌──────────────┐   ┌──────────────┐
│   Code   │ + │ Builder Image│ = │ Image finale │
│  Source  │   │  (S2I)       │   │ (Application)│
└──────────┘   └──────────────┘   └──────────────┘
    Git            CRI-O             Registry
```

**Avantages :**

- Pas besoin d'écrire de Dockerfile
- Images standardisées et sécurisées
- Builds reproductibles

# S2I — En pratique

```
# Déployer depuis GitHub avec S2I
oc new-app python:3.9-ubi8~https://github.com/sclorg/django-ex.git \
  --name=mon-app-django

# Format : <builder-image>~<repo-git>

# Suivre le build
oc logs -f bc/mon-app-django

# Voir le BuildConfig créé
oc get bc mon-app-django -o yaml

# Relancer un build
oc start-build mon-app-django

# Build depuis un répertoire local
oc start-build mon-app-django --from-dir=.
```

# BuildConfig — Configuration

```yaml
apiVersion: build.openshift.io/v1
kind: BuildConfig
metadata:
  name: mon-app-build
spec:
  source:
    type: Git
    git:
      uri: "https://github.com/monrepo/mon-app.git"
      ref: "main"
  strategy:
    type: Source
    sourceStrategy:
      from:
        kind: ImageStreamTag
        name: "python:3.9-ubi8"
        namespace: openshift
      env:
        - name: APP_CONFIG
          value: "production"
  output:
    to:
      kind: ImageStreamTag
      name: "mon-app:latest"
  triggers:
    - type: GitHub
      github:
```

# Stratégies de build

| Stratégie | Description | Usage |
|---|---|---|
| **Source (S2I)** | Build à partir du code source | Applications standard |
| **Docker** | Build depuis un Dockerfile | Contrôle total |
| **Pipeline** | Jenkinsfile/Tekton | CI/CD avancé |
| **Custom** | Builder personnalisé | Besoins spécifiques |

```
# Build Docker
oc new-build --strategy=docker \
  --name=mon-build-docker \
  https://github.com/monrepo/app-avec-dockerfile.git

# Build Pipeline (Tekton)
oc apply -f pipeline.yaml
```

# Stratégies de déploiement

```yaml
spec:
  strategy:
    type: RollingUpdate          # ou Recreate
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
```

| Stratégie | Description | Downtime |
|---|---|---|
| **Rolling** | Remplacement progressif | Non |
| **Recreate** | Arrêt puis redémarrage | Oui |
| **Blue-Green** | Deux versions en parallèle | Non |
| **Canary** | Déploiement partiel | Non |

# Module 5

## Gestion des Conteneurs et des Images

*Durée : 2 heures*

Utopios® Tous droits réservés - lot 4-002 20/03/2026

# ImageStreams

Les **ImageStreams** sont des références virtuelles vers des images :

```
# Lister les ImageStreams disponibles
oc get is -n openshift

# Créer un ImageStream
oc create imagestream mon-app

# Importer une image externe
oc import-image mon-app:latest \
  --from=docker.io/library/nginx:latest \
  --confirm

# Voir les tags disponibles
oc get istag
```

# Registre interne OpenShift

OpenShift intègre un registre d'images :

```
# Vérifier le registre
oc get pods -n openshift-image-registry

# Se connecter au registre interne
oc registry login

# Obtenir l'URL du registre
oc registry info

# Pousser une image vers le registre interne
# 1. Tag l'image
podman tag mon-app:local \
  default-route-openshift-image-registry.apps-crc.testing/mon-projet/mon-app:v1

# 2. Login au registre
podman login -u $(oc whoami) -p $(oc whoami -t) \
  default-route-openshift-image-registry.apps-crc.testing
```

# Construction d'images avec Dockerfile

```
# Dockerfile multi-stage optimisé pour OpenShift
FROM registry.access.redhat.com/ubi8/python-39:latest AS builder
WORKDIR /opt/app-root/src
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

FROM registry.access.redhat.com/ubi8/python-39:latest
WORKDIR /opt/app-root/src
COPY --from=builder /opt/app-root/src .

# Important : OpenShift exécute avec un UID aléatoire
```

# Bonnes pratiques pour les images OpenShift

**Sécurité :**

- Utiliser les images de base UBI (Universal Base Image)
- Ne jamais inclure de secrets dans l'image
- Exécuter en tant que non-root (`USER 1001`)
- Scanner les vulnérabilités avec `oc adm inspect`

**Performance :**

- Multi-stage builds pour réduire la taille

# ConfigMaps et Secrets

```
# ConfigMap depuis des valeurs littérales
oc create configmap app-config \
  --from-literal=DB_HOST=mysql \
  --from-literal=DB_PORT=3306 \
  --from-literal=APP_ENV=production

# ConfigMap depuis un fichier
oc create configmap nginx-config \
  --from-file=nginx.conf

# Secret
oc create secret generic db-credentials \
  --from-literal=username=admin \
  --from-literal=password=supersecret

# Utilisation dans un Deployment
oc set env deployment/mon-app --from=configmap/app-config
oc set env deployment/mon-app --from=secret/db-credentials
```

# Monter ConfigMaps et Secrets comme volumes

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mon-app
spec:
  template:
    spec:
      containers:
        - name: mon-app
          image: mon-app:latest
          volumeMounts:
            - name: config-volume
              mountPath: /etc/config
              readOnly: true
            - name: secret-volume
              mountPath: /etc/secrets
              readOnly: true
      volumes:
```

# Module 6

## Optimisation des Services et Microservices

*Durée : 2 heures*

# D'un monolithe aux microservices

```
Monolithe                          Microservices

┌──────────────────┐          ┌──────┐  ┌──────┐  ┌──────┐
│                  │          │ Auth │  │ API  │  │ UI   │
│  Application     │          │ Svc  │  │ Svc  │  │ Svc  │
│  complète        │    =>    ├──────┤  ├──────┤  ├──────┤
│                  │          │ Cart │  │Order │  │Notif │
│                  │          │ Svc  │  │ Svc  │  │ Svc  │
└──────────────────┘          └──────┘  └──────┘  └──────┘
```

**Avantages :**

- Déploiement indépendant de chaque service
- Scaling granulaire

# Pattern de décomposition

**Stratégie progressive :**

1. **Identifier** les domaines métier (DDD - Domain Driven Design)
2. **Extraire** les services les plus indépendants en premier
3. **Définir** les contrats d'API entre services
4. **Déployer** chaque service dans son propre Pod
5. **Connecter** via des Services Kubernetes

```
# Créer les services séparément
oc new-app nodejs~https://github.com/example/frontend.git \
  --name=frontend
```

# Communication inter-services

```yaml
# Service discovery via DNS interne
# Format : <service-name>.<namespace>.svc.cluster.local

# Exemple dans la configuration du frontend
apiVersion: v1
kind: ConfigMap
metadata:
  name: frontend-config
data:
  API_URL: "http://api-service.mon-projet.svc.cluster.local:8080"
  DB_HOST: "database.mon-projet.svc.cluster.local"
```

```bash
# Test de connectivité entre services
oc rsh frontend-pod -- curl http://api-service:8080/health
```

# Volumes persistants (PVC)

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-data
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: standard
```

```bash
# Créer le PVC
oc apply -f pvc-mysql.yaml
```

# Health Checks — Probes

```yaml
spec:
  containers:
    - name: mon-app
      image: mon-app:latest
      livenessProbe:
        httpGet:
          path: /healthz
          port: 8080
        initialDelaySeconds: 30
        periodSeconds: 10
        failureThreshold: 3
      readinessProbe:
        httpGet:
          path: /ready
          port: 8080
        initialDelaySeconds: 5
        periodSeconds: 5
      startupProbe:
        httpGet:
          path: /healthz
          port: 8080
```

# Autoscaling — HPA

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: mon-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: mon-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
```

# Services et découverte

```yaml
apiVersion: v1
kind: Service
metadata:
  name: api-service
spec:
  selector:
    app: api-service
  ports:
    - name: http
      port: 8080
      targetPort: 8080
  type: ClusterIP
---
apiVersion: v1
kind: Service
metadata:
  name: api-service-headless
spec:
  clusterIP: None
  selector:
    app: api-service
```